

Progress Report 5

Student: Michael Hein (300375535)

Course: CSIS 4495 – Applied Research Project

Project: Real-Time Cryptocurrency Streaming Pipeline

Reporting Period: Mar 18 – Mar 23, 2026

1. Work Date/Hours Logs

Date	Number of Hours	Description of Work Done
Mar 19, 2026	1.5	Redesigned dashboard header and branding — renamed to "CryptoStream", added Inter font (Google Fonts), new favicon, and lightning bolt logo. Restructured header controls layout with ARIA labels and keyboard shortcuts button.
Mar 19, 2026	1.5	Implemented Market Insights section — six summary cards showing market direction, average price change, most active symbol, highest volume, most volatile, and BTC dominance. Computed live from streaming metrics in <code>app.js</code> .
Mar 19, 2026	1	Built Watchlist/Favorites feature — users can star symbols in the metrics table, persisted to <code>localStorage</code> . Watchlist section renders live cards with price, change, trades, and volume. Clicking a card opens the detail modal.
Mar 19, 2026	1	Added direction filters (All / Gainers / Losers) above the metrics table and skeleton loading states for initial page load. Improved empty-state UX with icons and contextual messages.
Mar 19, 2026	1.5	Overhauled CSS — moved to Tailwind-inspired color palette, added collapsible chart sections with animated expand/collapse, section headers with icon badges, status bar dots, and responsive layout refinements. ~1,400 lines of CSS updated.
Mar 21, 2026	2	Created MLX LLM service (<code>src/utils/llm_service.py</code>) — <code>MLXClient</code> class wrapping the OpenAI-compatible API from <code>mlx_lm.server</code> . Supports non-streaming and SSE streaming chat completions, model health checks, and single-shot generation.
Mar 21, 2026	1.5	Built market context builder (<code>src/utils/context_builder.py</code>) — converts live dashboard payload (metrics, alerts, anomalies, arbitrage, exchange stats, recent trades) into structured text for LLM system prompt injection.
Mar 21, 2026	2	Added four AI API endpoints to <code>api_server.py</code> : <code>/api/ai/health</code> , <code>/api/ai/chat</code> (streaming SSE), <code>/api/ai/insights</code> (auto-generated summaries), and <code>/api/ai/query</code> (natural-language data queries with structured JSON responses).

Date	Number of Hours	Description of Work Done
Mar 21, 2026	1	Implemented background insight generation loop — a daemon thread periodically calls the LLM with a market summary prompt and caches the latest insight for the dashboard to fetch.
Mar 21, 2026	1	Added AI configuration to <code>src/utils/config.py</code> and <code>docker-compose.yml</code> — <code>AI_ENABLED</code> , <code>MLX_SERVER_URL</code> , <code>MLX_MODEL</code> , <code>AI_INSIGHT_INTERVAL_SEC</code> , <code>AI_REQUEST_TIMEOUT_SEC</code> . Used <code>host.docker.internal</code> for container-to-host LLM access.
Mar 21, 2026	0.5	Created <code>scripts/start_mlx_server.sh</code> — helper script to launch the MLX LM server on Apple Silicon, with auto-install of <code>mlx-lm</code> , configurable model and port, and architecture validation.
Mar 21, 2026	1	Integrated AI chat panel into the dashboard frontend — chat UI with streaming response rendering, message history, and market context injection. Added AI insights display to the Market Insights section.

Total Hours This Period: ~16

2. Summary Description of Work Done

This reporting period focused on two major areas: (1) a complete dashboard UI redesign and (2) integrating an AI assistant powered by a local LLM via Apple MLX.

Dashboard UI Redesign: I overhauled the entire dashboard frontend to improve usability and visual polish. The app was rebranded as "CryptoStream" with Inter font, a new favicon, and a Tailwind-inspired color system. Key new features include a Market Insights row (six live-computed summary cards for market direction, average change, most active symbol, highest volume, most volatile symbol, and BTC dominance), a Watchlist/Favorites system with `localStorage` persistence, direction filters (Gainers/Losers), collapsible chart sections, skeleton loading states, and improved empty states. The CSS was largely rewritten (~1,400 lines changed) for consistent theming, responsive layout, and smoother animations.

AI Assistant (MLX LLM Integration): I integrated a local LLM into the pipeline using Apple's MLX framework. The `MLXClient` class wraps the OpenAI-compatible API exposed by `mlx_lm.server`, supporting both synchronous and streaming (SSE) chat completions. A context builder module converts the live dashboard data (metrics, alerts, anomalies, arbitrage opportunities, exchange stats) into structured text injected into the LLM system prompt. Four new API endpoints were added: health check, streaming chat, auto-generated market insights, and natural-language data queries that return structured JSON. A background daemon thread periodically generates market insight summaries. The AI is fully configurable via environment variables and integrates with the Docker Compose setup using `host.docker.internal` networking.

One challenge was ensuring the Docker container could reach the MLX server running on the host machine — this was resolved by adding `extra_hosts: host.docker.internal:host-gateway` to the dashboard service in `docker-compose.yml`.

Next steps: Write unit tests for the AI modules (`llm_service.py`, `context_builder.py`), add error handling for LLM timeouts in the frontend, and prepare the final project report and presentation.

3. Repo Check-In of Implementation Completed

The following files/folders have been checked into the repo since the last progress report:

New files:

- `src/utils/llm_service.py` — MLX LLM client (health checks, chat, streaming, generation)
- `src/utils/context_builder.py` — Converts dashboard payload to structured LLM context text
- `scripts/start_mlx_server.sh` — Shell script to start the MLX LM server on Apple Silicon
- `Implementation/dashboard/web/favicon.svg` — New dashboard favicon

Modified files:

- `Implementation/dashboard/api_server.py` — Four new AI endpoints (`/api/ai/health`, `/api/ai/chat`, `/api/ai/insights`, `/api/ai/query`), background insight generation loop, AI state management
- `Implementation/dashboard/web/index.html` — Market Insights section, Watchlist section, direction filters, skeleton loading, collapsible sections, rebranding to CryptoStream, keyboard shortcuts button
- `Implementation/dashboard/web/js/app.js` — Favorites/watchlist logic with localStorage, market insights rendering, direction filtering, AI chat integration, updated color palette, improved sparklines and empty states
- `Implementation/dashboard/web/css/styles.css` — Complete visual overhaul (~1,400+ lines changed): Tailwind-inspired colors, Inter font, collapsible sections, status dots, section headers with icons, watchlist cards, market insight cards, responsive refinements
- `src/utils/config.py` — New AI config fields: `ai_enabled`, `mlx_server_url`, `mlx_model`, `ai_insight_interval_sec`, `ai_request_timeout_sec`
- `docker-compose.yml` — Dashboard service: added `extra_hosts` for `host.docker.internal`, AI environment variables (`MLX_SERVER_URL`, `MLX_MODEL`, `AI_ENABLED`, `AI_INSIGHT_INTERVAL_SEC`)

4. AI Use Section

AI Tool Name	Version, Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition
Cursor (Claude)	Claude 4.6 Opus / Premium	Dashboard UI redesign — layout structure, Market Insights cards, Watchlist feature	Designed the UX flow for favorites persistence; customized the color palette and responsive breakpoints; tested across screen sizes

AI Tool Name	Version, Account Type	Specific Feature for Which the AI Tool Was Used	Value Addition
Cursor (Claude)	Claude 4.6 Opus / Premium	MLX LLM service design — <code>MLXClient</code> class, SSE streaming, health checks	Chose MLX over Ollama for Apple Silicon performance; validated OpenAI-compatible API contract; added error handling and timeout logic
Cursor (Claude)	Claude 4.6 Opus / Premium	Context builder — converting dashboard payload to structured LLM-readable text	Decided on table format for metrics (vs. JSON) for better LLM reasoning; selected which data fields to include for relevance
Cursor (Claude)	Claude 4.6 Opus / Premium	AI API endpoints — chat streaming, insight generation loop, natural-language query parsing	Designed the background insight thread timing; implemented SSE streaming pattern; structured JSON fallback for query parsing
Cursor (Claude)	Claude 4.6 Opus / Premium	Docker networking — container-to-host MLX server connectivity	Researched <code>host.docker.internal</code> pattern; configured <code>extra_hosts</code> in Compose; validated on macOS Docker Desktop

5. Appendix: AI Prompt History

Prompt 1 — Dashboard UI redesign

"Redesign the dashboard with a modern look — add Google Fonts (Inter), rebrand to CryptoStream, add market insight summary cards (market direction, avg change, most active, highest volume, most volatile, BTC dominance), and a watchlist feature where users can star symbols. Use a Tailwind-inspired color palette."

Prompt 2 — Direction filters and skeleton loading

"Add direction filter buttons (All, Gainers, Losers) above the metrics table. Also add skeleton loading rows when the table is waiting for data, and improve the empty states with icons and helpful messages."

Prompt 3 — Collapsible chart sections

"Make the chart sections collapsible — clicking the section header should expand/collapse the charts with a smooth animation. Add section icons and update the CSS."

Prompt 4 — MLX LLM integration

"I want to add an AI assistant to the dashboard using Apple MLX. Create an LLM service that connects to `mlx_lm.server` (OpenAI-compatible API), supports streaming chat, and can generate market insights from the live dashboard data. Add API endpoints for chat, insights, and natural-language data queries."

Prompt 5 — Context builder for LLM

"Build a module that converts the dashboard payload (metrics, alerts, anomalies, arbitrage, exchange stats, recent trades) into a structured text format that I can inject into the LLM system prompt as market context."

Prompt 6 — Docker container to host MLX server

"The dashboard runs in Docker but the MLX server runs on the host. How do I connect the container to localhost:8080 on the host? I'm on macOS with Docker Desktop."

Prompt 7 — Background insight generation

"Add a background thread to the dashboard that periodically generates market insight summaries using the LLM and caches them. The dashboard should be able to fetch the latest insight via an API endpoint."